Learning Objectives

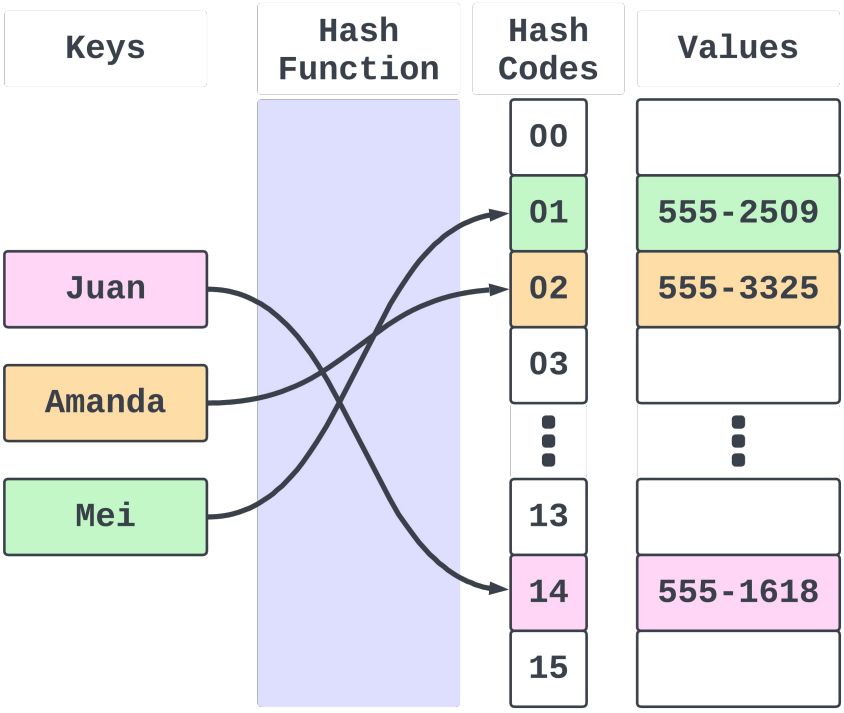
Learners will be able to…

- **Implement quadratic hashing**
- **Use double hashing to avoid secondary clustering**
- **Rehash a table when the load factor is too big**
- **Evaluate and select appropriate hashing strategies**

Improving Hash Table Performance

Fine-Tuning Hash Tables

In this assignment, we delve deeper into the world of hash tables and explore the advanced techniques that make them one of the most efficient and widely used data structures in computer science. Hash tables are fundamental to a plethora of applications, ranging from database indexing to caching. Their performance, however, hinges on several factors including the efficiency of the hash function, the handling of collisions, and the dynamic resizing of the hash table.



Performance is a critical aspect of hash tables. It's influenced by:

- **Load Factor**: The ratio of the number of stored entries to the number of buckets. A high load factor can increase the chances of collisions, leading to a decline in performance.
- **Hash Function**: A good hash function distributes keys uniformly across the table, minimizing collisions and ensuring consistent performance.
- **Collision Resolution Strategy**: How the hash table handles collisions can significantly impact its performance. Strategies like chaining, linear probing, quadratic probing, and double hashing offer different trade-offs in terms of complexity and performance.

We will cover a range of techniques to optimize hash table performance:

- **Quadratic Probing**: Similar to linear probing but addresses the issue of clustering by using a quadratic function to calculate the probe sequence.
- **Rehashing**: Involves resizing the hash table and rehashing all existing keys when the load factor crosses a certain threshold.
- **Double Hashing**: Utilizes a second hash function to determine the probe sequence, significantly reducing the chance of clustering and collisions.

Understanding and implementing these hashing techniques and performance optimization strategies are crucial for any software developer or computer scientist. The following pages will provide a detailed exploration of each topic, practical coding exercises, and real-world applications to reinforce your learning and enable you to harness the full potential of hash tables in your projects.

Quadratic Probing

Improving Linear Probing

We previously saw how linear probing can handle hashing collisions. However, there are some issues with this resolution strategy. Linear probing works by finding a slot in the table. If it is full, it advances to the next available slot in a linear manner. That means, by definition, linear probing is going to create clusters of used slots around generated hash indices. This is called primary clustering.

A good hash table will reduce primary clustering (as much as possible). This is partially done through tweaking the hashing function, but the collision resolution strategy also plays a role. Quadratic probing is an open addressing collision resolution technique that addresses some limitations of linear probing, particularly primary clustering. In quadratic probing, the interval between probes increases quadratically.

Key Concepts:

- **Probing Function:** The function used in quadratic probing is typically of the form `(c1 * i^2 + c2 * i) % table_size`, where `i` is the i-th probe.
- **Secondary Clustering:** Less severe than primary clustering but still a factor, where certain slots become more likely to be probed than others.

Implementation Notes:

- Choose the constants `c1` and `c2` carefully to ensure a good distribution of probe sequences.
- Similar to linear probing, implement wrap-around at the end of the table.

Unlike linear probing, where each probe is one slot away from the previous one, quadratic probing uses a hash function that adds a quadratic polynomial to the index. This method reduces the likelihood of clustering, spreading out any clusters that happen to form.

The probe sequence for quadratic probing is defined as follows:

```
hashIndex = (hashFunction(key) + c1 * i + c2 * i^2) % tableSize
```

Where `i` is the i-th probe, and `c1` and `c2` are constants.

Implementing Quadratic Probing

Start by creating the class for quadratic probing. We are going to use the `int` data type for the keys and values so that `-1` can signify an empty slot in the hash table. The hash function will remain the simple one we have seen in the past.

```
class QuadraticProbingHashTable {
    private:
        int maxSize;
        int* keys;
        int* values;

    public:
        // Constructor
        QuadraticProbingHashTable(int size) {
            maxSize = size;
            keys = new int[maxSize];
            values = new int[maxSize];
            for (int i = 0; i < maxSize; i++) {
                keys[i] = -1;
                values[i] = -1;
            }
        }

        int hashFunction(int key) {
            return key % maxSize;
        }

};
```

Since we are going to perform quadratic probing for each operation, we are going to separate this out into its own method. Using the formula above, we are going to create the following quadratic probing method where `c1` has the value `17` and `c2` has the value `23`.

```
        int quadProbing(int key, int i) {
            return (hashFunction(key) + 17 * i + 23 * i * i) %
        maxSize;
        }
```

When inserting key-value pairs, start by generating the hash index. We also need to create `i` to keep track of the number of probes. If the slot at the generated slot is not empty, start the quadratic probing. To keep things simple, we are going to use `0` as the value of `c1` and `1` as the value of `c2` in the quadratic probing formula. Once you have found an empty slot, place the given `key` and `value` into their respective array.

```
    void put(int key, int value) {
        int hashIndex = hashFunction(key);
        int i = 0; // probe number

        while (keys[hashIndex] != -1 && keys[hashIndex] !=
    key) {
            hashIndex = quadProbing(key, i);
            i++;
        }

        keys[hashIndex] = key;
        values[hashIndex] = value;
    }
```

Searching for a key-value pair works in a similar manner. Generate the hash index and set the probe counter to `0`. If the slot is empty (`-1`) return `-1` since the key is not in the hash table. If the slot is occupied and the slot contains the desired key, return the value. Otherwise, start the quadratic probing process.

```
    int get(int key) {
        int hashIndex = hashFunction(key);
        int i = 0; // probe number

        while (keys[hashIndex] != -1) {
            if (keys[hashIndex] == key) {
                return values[hashIndex];
            }
            hashIndex = quadProbing(key, i);
            i++;
        }

        return -1; // Key not found
    }
```

Finally, to delete a key-value pair, generate the hash index and keep track of the probing. If the slot is not empty and the stored key matches the given key, set the elements at `hashIndex` in both the `keys` and `values` arrays to `-1` to show that the slot is now empty. Otherwise, start the quadratic probing.

```
    void remove(int key) {
        int hashIndex = hashFunction(key);
        int i = 0; // probe number

        while (keys[hashIndex] != -1) {
            if (keys[hashIndex] == key) {
                keys[hashIndex] = -1;
                values[hashIndex] = -1;
                return;
            }
            hashIndex = quadProbing(key, i);
            i++;
        }
    }
```

To test our code, create the `main` method. Instantiate a `QuadraticProbingHashTable` object and be sure to use a prime number as the size. Add three key-value pairs to the table to test the `put` method. Test the `get` method by printing the values for the given keys. Delete a key-value pair and try to print this pair to test the `remove` method.

```
int main() {
    QuadraticProbingHashTable hashTable(7);

    // Inserting key-value pairs
    hashTable.put(1, 100);
    hashTable.put(2, 200);
    hashTable.put(11, 300);

    // Retrieving values
    std::cout << "Value for key 1: " << hashTable.get(1)
  << std::endl;
    std::cout << "Value for key 2: " << hashTable.get(2)
  << std::endl;
    std::cout << "Value for key 11: " << hashTable.get(11)
  << std::endl;

    // Deleting a key
    hashTable.remove(2);
    std::cout << "Value for key 2 after deletion: " <<
  hashTable.get(2) << std::endl;

    return 0;
}
```

▼ Code

Your code should look like this:

```
#include <iostream>

class QuadraticProbingHashTable {
    private:
        int maxSize;
        int* keys;
```

```
    int* values;

public:
    // Constructor
    QuadraticProbingHashTable(int size) {
        maxSize = size;
        keys = new int[maxSize];
        values = new int[maxSize];
        for (int i = 0; i < maxSize; i++) {
            keys[i] = -1;
            values[i] = -1;
        }
    }

    int hashFunction(int key) {
        return key % maxSize;
    }

    int quadProbing(int key, int i) {
        return (hashFunction(key) + 17 * i + 23 * i * i) %
    maxSize;
    }

    void put(int key, int value) {
        int hashIndex = hashFunction(key);
        int i = 0; // probe number

        while (keys[hashIndex] != -1 && keys[hashIndex] !=
    key) {
            hashIndex = quadProbing(key, i);
            i++;
        }

        keys[hashIndex] = key;
        values[hashIndex] = value;
    }

    int get(int key) {
        int hashIndex = hashFunction(key);
        int i = 0; // probe number

        while (keys[hashIndex] != -1) {
            if (keys[hashIndex] == key) {
                return values[hashIndex];
            }
            hashIndex = quadProbing(key, i);
            i++;
        }

        return -1; // Key not found
    }

    void remove(int key) {
        int hashIndex = hashFunction(key);
        int i = 0; // probe number

        while (keys[hashIndex] != -1) {
            if (keys[hashIndex] == key) {
```

```
                    keys[hashIndex] = -1;
                    values[hashIndex] = -1;
                    return;
                }
                hashIndex = quadProbing(key, i);
                i++;
            }
        }
};

int main() {
    QuadraticProbingHashTable hashTable(7);

    // Inserting key-value pairs
    hashTable.put(1, 100);
    hashTable.put(2, 200);
    hashTable.put(11, 300);

    // Retrieving values
    std::cout << "Value for key 1: " << hashTable.get(1) <<
        std::endl;
    std::cout << "Value for key 2: " << hashTable.get(2) <<
        std::endl;
    std::cout << "Value for key 11: " << hashTable.get(11) <<
        std::endl;

    // Deleting a key
    hashTable.remove(2);
    std::cout << "Value for key 2 after deletion: " <<
        hashTable.get(2) << std::endl;

    return 0;
}
```

You should see the following output:

```
Value for key 1: 100
Value for key 2: 200
Value for key 11: 300
Value for key 2 after deletion: -1
```

Quadratic probing offers a more sophisticated approach to collision resolution compared to linear probing. By using a quadratic function to determine the probe sequence, quadratic probing reduces the issue of primary clustering, leading to better performance in scenarios where collisions are common.

Secondary Clustering

Understanding Secondary Clustering

Secondary clustering occurs in quadratic probing when different keys follow the same probe sequence, leading to potential clustering issues. Although less severe than primary clustering, it's a factor to consider in highly populated tables.

Let's take a look at a simple example of secondary clustering. The code in the editor to the left creates a hash table with a maximum size of `10`. The quadratic probing algorithm uses `0` for `c1` and `1` for `c2`. Essentially, these constants do not factor into the end result:

```
private:
    int quadProbing(int key, int i) {
        return (hashFunction(key) + i * i) % maxSize;
    }
```

We are going to add seven keys (2, 12, 22, 32, 42, 52, and 62) to the hash table. The `put` method has an extra print statement to print the calculated hash index. You will see a repetition of numbers. **Important**, the loop is going to run over 46,000 times and then eventually terminate. We will discuss why this is below.

Key	Probing	Hashed Index
2	$2 + 0^2 \mod 10$	2
12	$2 + 1^2 \mod 10$	3
22	$2 + 2^2 \mod 10$	6
32	$2 + 3^2 \mod 10$	1
42	$2 + 4^2 \mod 10$	8
52	$2 + 5^2 \mod 10$	7

The next key to insert in the hash table is 62. If we plug this into our quadratic probing formula, we get $2 + 6^2 = 38$. The next step is to perform the modulo operation, $38 \mod 10 = 8$. However, index `8` is already taken (key `42`). So the loop continues. The next iteration returns `1`, which is taken (key `32`). The next iteration returns `6`, which is taken (key `22`), and so on. The hash table is not full, but the quadratic formula continually returns a result of a slot that is already taken.

▼ **Why is the loop not infinite?**

> In C++, integer overflow is an undefined behavior. Most implementations will simply wrap around. This means large numbers eventually cycle back to small numbers. If you were to run the same program in Java, you would get a timeout error.

While quadratic probing improves upon the distribution of keys when compared to linear probing, it introduces some secondary clustering. There are ways of dealing with this. For starters, using quality values for the `c1` and `c2` constants can help avoid this situation. Common solutions entail resizing the hash table or using double hashing. We will discuss both of those topics over the next few pages.

For now, we are going to set a stop condition. The easiest stop condition to implement is counting the number of probes. When these probes exceed the maximum size of the hash table then one of two things is true – either the table is full or the quadratic formula is stuck in an infinite loop.

Update the `put` method to count the number of probes during each iteration of the while loop. Then ask if `probeCount` is greater than `maxSize`. If so, throw an exception. We do not want to simply exit the method. Without any feedback, we would assume that the key-value pair was successfully added to the hash table even though it was not. Throw a runtime error saying that the table is full or that a wrapping issues has occurred.

```
    void put(int key, int value) {
        int hashIndex = hashFunction(key);
        int i = 0; // probe number
        int probeCount = 0;

        while (keys[hashIndex] != -1 && keys[hashIndex] != key)
        {
            hashIndex = quadProbing(key, i);
            i++;
            probeCount++;

            if (probeCount > maxSize) {
                throw std::runtime_error("Table is full or
        wrapping issue occurred");
            }
        }

        keys[hashIndex] = key;
        values[hashIndex] = value;
    }
```

▼ Code

> Your code should look like this:

```
#include <iostream>
#include <cmath>
```

```cpp
#include <stdexcept>

class SecondaryClustering {
private:
    int maxSize;
    int* keys;
    int* values;

public:
    SecondaryClustering(int size) {
        maxSize = size;
        keys = new int[maxSize];
        values = new int[maxSize];
        for (int i = 0; i < maxSize; ++i) {
            keys[i] = -1;
            values[i] = -1;
        }
    }

    ~SecondaryClustering() {
        delete[] keys;
        delete[] values;
    }

    int hashFunction(int key) {
        return key % maxSize;
    }

    int quadProbing(int key, int i) {
        return (hashFunction(key) + i * i) % maxSize;
    }

    void put(int key, int value) {
        int hashIndex = hashFunction(key);
        int i = 0; // probe number
        int probeCount = 0;

        while (keys[hashIndex] != -1 && keys[hashIndex] !=
        key) {
            hashIndex = quadProbing(key, i);
            i++;
            probeCount++;

            if (probeCount > maxSize) {
                throw std::runtime_error("Table is full or
        wrapping issue occurred");
            }
        }

        keys[hashIndex] = key;
        values[hashIndex] = value;
    }

    int get(int key) {
        int hashIndex = hashFunction(key);
        int i = 0; // probe number

        while (keys[hashIndex] != -1) {
```

```
            if (keys[hashIndex] == key) {
                return values[hashIndex];
            }
            hashIndex = quadProbing(hashIndex, i);
            i++;
        }

        return -1; // Key not found
    }

    void remove(int key) {
        int hashIndex = hashFunction(key);
        int i = 0; // probe number

        while (keys[hashIndex] != -1) {
            if (keys[hashIndex] == key) {
                keys[hashIndex] = -1;
                values[hashIndex] = -1;
                return;
            }
            hashIndex = quadProbing(hashIndex, i);
            i++;
        }
    }

};

int main() {
    SecondaryClustering hashTable(10);
    hashTable.put(2, 2);
    hashTable.put(12, 2);
    hashTable.put(22, 2);
    hashTable.put(32, 2);
    hashTable.put(42, 2);
    hashTable.put(52, 2);
    hashTable.put(62, 2);
}
```

Run the program once again. The program still does not run properly, as it throws an exception when secondary clustering occurs. However, at least we get some feedback about what is not working. On the next page, we will discuss double hashing, a collision resolution strategy that can help us avoid the pitfalls of secondary clustering.

You should see the following output:

```
terminate called after throwing an instance of
        'std::runtime_error'
  what():  Table is full or wrapping issue occurred
.guides/bg.sh: line 3:   190 Aborted                 (core
        dumped) $5 $6
```

Quadratic probing is a powerful technique for collision resolution in hash tables. By understanding its intricacies and properly managing secondary clustering, table size, and the wrapping issue, you can optimize your hash table's performance for a wide range of applications. Remember that the choice of collision resolution strategy should be informed by the specific requirements and characteristics of your data and application.

Double Hashing

Dealing with Clustering

We have seen how both linear and quadratic probing can lead to clustering of hash codes. In some cases, this can lead to a near infinite loop. Let's turn our attention to double hashing. Double hashing is a sophisticated technique that minimizes the clustering effect more effectively than linear or quadratic probing. It uses two hash functions instead of one, providing a unique way to calculate probe sequences for collision resolution.

Key Concepts:

- **Second Hash Function**: This function should be independent of the first and designed to return a non-zero value.
- **Probing Sequence**: The sequence is calculated with the formula below. The variable `i` represents the i-th probe.

```
(hash1(key) + i * hash2(key)) % table_size
```

Implementation Notes:

- Ensure that the second hash function is as efficient and uniformly distributed as the first.
- Handle the case where the second hash function might return zero to avoid a near infinite loop.

Double hashing employs two distinct hash functions. The first hash function determines the initial position, and if a collision occurs, the second hash function calculates the interval for subsequent probes.

There is a slight chance that we could get a negative number while calculating the double hash. To avoid this, we are going to use something called floor modulo. Floor modulo is an operation that ensures a consistent, mathematically well-defined modulo result for negative numbers. Our code will use the method below when calculating the hash index.

```
int floorMod(int x, int y) {
    int r = x % y;
    return r < 0 ? r + y : r;
}
```

Double hashing significantly reduces both primary and secondary clustering. In particular, the second hashing function ensures that probe sequences are distributed more uniformly.

Double Hashing Implementation

Let's implement the same methods (`put`, `get`, and `remove`) for a hash table using double hashing. Start by declaring the `DoubleHashing` class. Just as before, this class has a maximum size and arrays for the keys and values stored in the table. Create two hashing methods. The first one will be the same hash function that we have been using. The second hashing function will subtract `1` from `maxSize` before performing the modulo operation. Notice how the second hashing function starts off with `1 + ...`. This ensures that if the rest of the calculation resolves to `0`, the return value will be non-zero. We are also going to add the `floorMod` method to ensure our hash index is positive.

```
class DoubleHashing {
private:
    int maxSize;
    int* keys;
    int* values;

    int hashFunction1(int key) {
        return key % maxSize;
    }

    int hashFunction2(int key) {
        return 1 + (key % (maxSize - 1));
    }

    int floorMod(int x, int y) {
        int r = x % y;
        return r < 0 ? r + y : r;
    }

};
```

Then comes the public methods for the `DoubleHashing` class. Start with the constructor.

```
public:
    DoubleHashing(int size) {
        maxSize = size;
        keys = new int[maxSize];
        values = new int[maxSize];
        for (int i = 0; i < maxSize; ++i) {
            keys[i] = -1;
            values[i] = -1;
        }
    }
```

In the `put` method, calculate the hash code by calling the first hashing function. You do not need to perform double hashing at this moment. If the current hash code is taken the second hashing function is used.

```
    void put(int key, int value) {
        int hashIndex = hashFunction1(key);
        int i = 0;

        while (keys[hashIndex] != -1 && keys[hashIndex] != key)
        {
            hashIndex = floorMod((hashFunction1(key) + i *
        hashFunction2(key)), maxSize);
            i++;
        }

        keys[hashIndex] = key;
        values[hashIndex] = value;
    }
```

Similarly, the `get` method will use the first hash function in the beginning. If a collision needs to be resolved, the second hash function will be called.

```
    int get(int key) {
        int hashIndex = hashFunction1(key);
        int i = 0;

        while (keys[hashIndex] != -1) {
            if (keys[hashIndex] == key) {
                return values[hashIndex];
            }
            hashIndex = floorMod((hashFunction1(key) + i *
        hashFunction2(key)), maxSize);
            i++;
        }

        return -1;
    }
```

Finally create the `remove` method that makes use of double hashing. You will only call the second hash function if there is a collision and the current slot does not contain the given key.

```
    void remove(int key) {
        int hashIndex = hashFunction1(key);
        int i = 0; // probe number

        while (keys[hashIndex] != -1) {
            if (keys[hashIndex] == key) {
                keys[hashIndex] = -1;
                values[hashIndex] = -1;
                return;
            }
            hashIndex = floorMod((hashFunction1(key) + i *
        hashFunction2(key)), maxSize);
            i++;
        }
    }
```

Testing the Code

Let's test our double hashing implementation by inserting, retrieving, and deleting keys-value pairs:

```
int main() {
    DoubleHashing hashTable(10);
    hashTable.put(1, 100);
    hashTable.put(2, 200);
    hashTable.put(11, 300);

    // Retrieving values
    std::cout << "Value for key 1: " << hashTable.get(1) <<
        std::endl;
    std::cout << "Value for key 2: " << hashTable.get(2) <<
        std::endl;
    std::cout << "Value for key 11: " << hashTable.get(11) <<
        std::endl;

    // Deleting a key
    hashTable.remove(2);
    std::cout << "Value for key 2 after deletion: " <<
        hashTable.get(2) << std::endl;

    return 0;
}
```

▼ **Code**

Your code should look like this:

```


#include <iostream>


class DoubleHashing {
private:
    int maxSize;
    int* keys;
    int* values;

    int hashFunction1(int key) {
        return key % maxSize;
    }

    int hashFunction2(int key) {
        return 1 + (key % (maxSize - 1));
    }

    int floorMod(int x, int y) {
        int r = x % y;
        return r < 0 ? r + y : r;
    }

public:
```

```
    DoubleHashing(int size) {
        maxSize = size;
        keys = new int[maxSize];
        values = new int[maxSize];
        for (int i = 0; i < maxSize; ++i) {
            keys[i] = -1;
            values[i] = -1;
        }
    }

    void put(int key, int value) {
        int hashIndex = hashFunction1(key);
        int i = 0;

        while (keys[hashIndex] != -1 && keys[hashIndex] != key)
        {
            hashIndex = floorMod((hashFunction1(key) + i *
        hashFunction2(key)), maxSize);
            i++;
        }

        keys[hashIndex] = key;
        values[hashIndex] = value;
    }

    int get(int key) {
        int hashIndex = hashFunction1(key);
        int i = 0;

        while (keys[hashIndex] != -1) {
            if (keys[hashIndex] == key) {
                return values[hashIndex];
            }
            hashIndex = floorMod((hashFunction1(key) + i *
        hashFunction2(key)), maxSize);
            i++;
        }

        return -1;
    }

    void remove(int key) {
        int hashIndex = hashFunction1(key);
        int i = 0; // probe number

        while (keys[hashIndex] != -1) {
            if (keys[hashIndex] == key) {
                keys[hashIndex] = -1;
                values[hashIndex] = -1;
                return;
            }
            hashIndex = floorMod((hashFunction1(key) + i *
        hashFunction2(key)), maxSize);
            i++;
        }
    }

};
```

```
int main() {
    DoubleHashing hashTable(10);
    hashTable.put(1, 100);
    hashTable.put(2, 200);
    hashTable.put(11, 300);

    // Retrieving values
    std::cout << "Value for key 1: " << hashTable.get(1) <<
        std::endl;
    std::cout << "Value for key 2: " << hashTable.get(2) <<
        std::endl;
    std::cout << "Value for key 11: " << hashTable.get(11) <<
        std::endl;

    // Deleting a key
    hashTable.remove(2);
    std::cout << "Value for key 2 after deletion: " <<
        hashTable.get(2) << std::endl;

    return 0;
}
```

You should see the following output:

```
Value for key 1: 100
Value for key 2: 200
Value for key 11: 300
Value for key 2 after deletion: -1
```

challenge

Try this variation

Let's update the code to insert the key-value pairs from the previous page that caused a near infinite loop due to secondary clustering. Start by adding a print statement to the `put` method that prints the value of `hashIndex`.

```
    void put(int key, int value) {
        int hashIndex = hashFunction1(key);
        int i = 0;

        while (keys[hashIndex] != -1 && keys[hashIndex] !=
        key) {
            hashIndex = floorMod((hashFunction1(key) + i *
        hashFunction2(key)), maxSize);
            i++;
        }
        std::cout << hashIndex << std::endl;
        keys[hashIndex] = key;
        values[hashIndex] = value;
    }
```

Then update `main` to insert the seven key-value pairs from the previous page.

```
int main() {
    DoubleHashing hashTable(10);
    hashTable.put(2, 2);
    hashTable.put(12, 2);
    hashTable.put(22, 2);
    hashTable.put(32, 2);
    hashTable.put(42, 2);
    hashTable.put(52, 2);
    hashTable.put(62, 2);

    return 0;
}
```

▼ What is happening?

> You should see seven different hashing indices. Because we are using double hashing, we can avoid the near infinite loop caused by secondary clustering associated with quadratic probing.

Double hashing stands out as an efficient collision resolution technique by offering a second layer of hashing, which significantly reduces the chances of clustering and collision. Its effectiveness hinges on the careful selection of two compatible hash functions. By mastering double hashing, you

enhance your ability to design robust and efficient hash tables, suitable for high-performance applications where minimizing collisions and clustering is paramount. As with any advanced data structure technique, understanding its characteristics and limitations is key to leveraging its full potential in real-world scenarios.

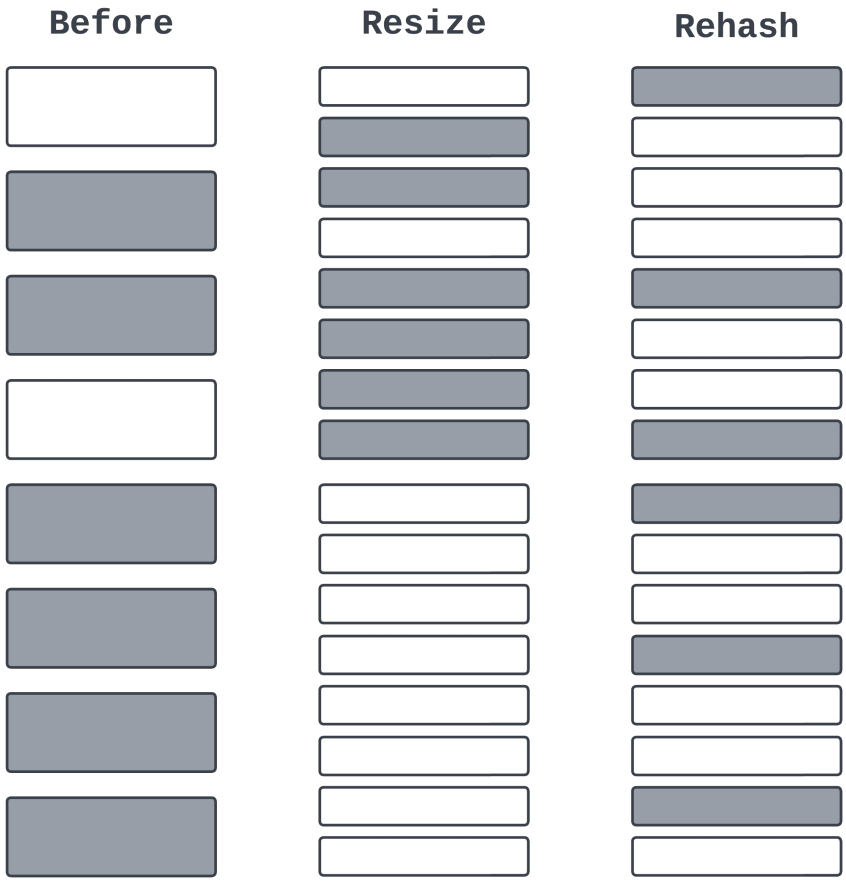
Rehashing

Optimal Table Size

One way to avoid collisions is to simply increase the size of the hash table. A larger capacity will change the results of hashing functions, which should increase the distribution of hash codes. However, there needs to be a balance. You do not want to initially allocate an arbitrarily large amount of memory. Resizing should be done incrementally and only when there is a performance improvement to be had.

This process is referred to as rehashing because this is a two-step process. First, create a larger hash table. Second, rehash all of the existing key-value pairs in the table. Both of these steps are necessary to make the most efficient hash table.

Use the image below as an example. The “Before” column represents the state of the hash table before any changes are made. It is almost full and there is a clustering problem. If we simply double the size of the table (the second “Resize” column) we have many more open slots in the hash table. However, we still have the clustering problem. The best solution is the third column (“Rehash”) because we get the extra capacity and the existing key-value pairs are redistributed to reduce any clustering problems.



The table size plays a crucial role in the performance of a hash table with quadratic probing. A load factor that is too high can lead to increased collisions and longer probe sequences, while a load factor that is too low can result in wasted space. A load factor of `0.7` is commonly used as the threshold for rehashing.

Rehashing a Table

You have been given some code that contains most of a hash table. We are going to add some methods to insert a key-value pair and rehash if necessary. Before we begin, it is important to note a few changes. The `Rehashing` class has the attributes `capacity` and `resizeThreshold`. `capacity` represents the current capacity (number of used slots) in the hash table, while `resizeThreshold` represents the limit at which the table should be resized. This hash table uses double hashing, and the first hash function multiplies the key by `31` before performing modulo.

Create the `put` method. The very first thing this method should do is check to see if the load factor is greater than the threshold for resizing. If so, call the `rehash` method. Otherwise, this is going to be very similar to the `put`

method covered in the double hashing section. However, there is one important difference. Be sure to increment `capacity` by `1` every time a key-value pair is added.

```
    void put(int key, int value) {
        if (loadFactor() >= resizeThreshold) {
            rehash();
        }

        int hashIndex = hashFunction1(key);
        int i = 0;

        while (keys[hashIndex] != nullptr && *keys[hashIndex] !=
        key) {
            hashIndex = (hashFunction1(key) + i *
        hashFunction2(key)) % maxSize;
            i++;
        }

        keys[hashIndex] = new int(key);
        values[hashIndex] = new int(value);
        capacity++;
    }
```

To calculate the load factor, we are going to return the current capacity of the table divided by the maximum size. Be sure to cast this as a double. Otherwise, C++ will return the closest integer (which will most likely be `0`).

```
    double loadFactor() const {
        return static_cast<double>(capacity) / maxSize;
    }
```

The `rehash` function saves the current size of the hash table in `oldSize` and then calculates a new size using the formula `maxSize * 2 + 3`. This doubles the capacity and adds a small constant to help reduce collisions.

Using the new size, new arrays, `keys` and `values`, are created. These will hold the rehashed keys and values.
Next, the algorithm iterates through the old arrays. For each non-null entry, it calculates a new index using the hash function `hashFunction1` and stores the `key` and `value` at that index in the new arrays.

Finally, the old arrays `oldKeys` and `oldValues` are deleted to free the allocated memory, ensuring there are no memory leaks.

```
    void rehash() {
        int oldSize = maxSize;
        maxSize = maxSize * 2 + 3;
        int** oldKeys = keys;
        int** oldValues = values;

        keys = new int*[maxSize]();
        values = new int*[maxSize]();

        for (int i = 0; i < oldSize; ++i) {
            if (oldKeys[i] != nullptr) {
                int newIndex = hashFunction1(*oldKeys[i]);
                keys[newIndex] = oldKeys[i];
                values[newIndex] = oldValues[i];
            }
        }

        delete[] oldKeys;
        delete[] oldValues;
    }
```

Testing the Code

To test our code, we are going to create a hash table with a maximum size of 4. Then add three key-value pairs. The `printState` method prints out the values stored in the keys and values arrays. This will help us visualize the changes being made to the hash table. Call `printState` to see what the table looks like before rehashing. Then add another key-value pair. Print the state one more time.

```
    int main() {
        Rehashing hashTable(4);

        hashTable.put(1, 11);
        hashTable.put(2, 22);
        hashTable.put(3, 33);

        std::cout << "Before" << std::endl;
        std::cout << "---------------" << std::endl;
        hashTable.printState();

        hashTable.put(4, 44);
        std::cout << "After" << std::endl;
        std::cout << "---------------" << std::endl;
        hashTable.printState();

        return 0;
    }
```

▼ **Code**

> Your code should look like this:

```cpp
#include <iostream>
#include <cstring>

class Rehashing {
private:
    int maxSize;
    int** keys;
    int** values;
    double resizeThreshold;
    int capacity;

public:
    Rehashing(int size) {
        maxSize = size;
        keys = new int*[maxSize]();
        values = new int*[maxSize]();
        resizeThreshold = 0.70;
        capacity = 0;
    }

    ~Rehashing() {
        for (int i = 0; i < maxSize; ++i) {
            delete keys[i];
            delete values[i];
        }
        delete[] keys;
        delete[] values;
    }

    int hashFunction1(int key) const {
        return (key * 31) % maxSize;
    }

    int hashFunction2(int key) const {
        return 1 + (key % (maxSize - 1));
    }

    void put(int key, int value) {
        if (loadFactor() >= resizeThreshold) {
            rehash();
        }

        int hashIndex = hashFunction1(key);
        int i = 0;

        while (keys[hashIndex] != nullptr &&
        *keys[hashIndex] != key) {
            hashIndex = (hashFunction1(key) + i *
        hashFunction2(key)) % maxSize;
            i++;
        }

        keys[hashIndex] = new int(key);
        values[hashIndex] = new int(value);
        capacity++;
    }
```

```
    double loadFactor() const {
        return static_cast<double>(capacity) / maxSize;
    }

    void rehash() {
        int oldSize = maxSize;
        maxSize = maxSize * 2 + 3;
        int** oldKeys = keys;
        int** oldValues = values;

        keys = new int*[maxSize]();
        values = new int*[maxSize]();

        for (int i = 0; i < oldSize; ++i) {
            if (oldKeys[i] != nullptr) {
                int newIndex = hashFunction1(*oldKeys[i]);
                keys[newIndex] = oldKeys[i];
                values[newIndex] = oldValues[i];
            }
        }

        delete[] oldKeys;
        delete[] oldValues;
    }

    void printState() const {
        std::cout << "Keys:   [";
        for (int i = 0; i < maxSize; i++) {
            if (keys[i] != nullptr) {
                std::cout << *keys[i];
            } else {
                std::cout << "null";
            }
            if (i < maxSize - 1) std::cout << ", ";
        }
        std::cout << "]" << std::endl;

        std::cout << "Values: [";
        for (int i = 0; i < maxSize; i++) {
            if (values[i] != nullptr) {
                std::cout << *values[i];
            } else {
                std::cout << "null";
            }
            if (i < maxSize - 1) std::cout << ", ";
        }
        std::cout << "]" << std::endl << std::endl;
    }
};

int main() {
    Rehashing hashTable(4);

    hashTable.put(1, 11);
    hashTable.put(2, 22);
    hashTable.put(3, 33);
```

```
    std::cout << "Before" << std::endl;
    std::cout << "---------------" << std::endl;
    hashTable.printState();

    hashTable.put(4, 44);
    std::cout << "After" << std::endl;
    std::cout << "---------------" << std::endl;
    hashTable.printState();

    return 0;
}
```

You should see the following output:

```
Before
---------------
Keys:   [null, 3, 2, 1]
Values: [null, 33, 22, 11]

After
---------------
Keys:   [null, null, null, 4, null, 3, null, 2, null, 1, null]
Values: [null, null, null, 44, null, 33, null, 22, null, 11,
        null]
```

Rehashing is an effective way to alleviate potential problems related to clustering. However, you should be mindful of the computational costs associated with resizing a hash table. This is especially true as hash tables start to grow in size. Resizing should only happen once a threshold has been crossed. Importantly, do not simply copy the key-value pairs from the old hash table into the new hash table. It is more than likely that the new hash code will be different from the old one. Rehash each of the keys before inserting the key-value pair into the resized hash table.

The Right Collision Resolution Strategy

Choosing the Right Collision Resolution

Having explored linear probing, quadratic probing, and double hashing, it's crucial to understand how to evaluate and choose the right collision resolution strategy for your specific use case. Each method has its strengths and weaknesses, and the optimal choice depends on factors like the expected load factor, the pattern and distribution of the keys, and the performance requirements of your application.

Evaluating Collision Resolution Techniques

When evaluating different collision resolution techniques, consider the following factors:

- **Performance in Terms of Time Complexity**: How do the average and worst-case time complexities of search, insert, and delete operations compare?
- **Clustering Effects**: Does the technique suffer from primary or secondary clustering, and how does that affect performance?
- **Load Factor Management**: How well does the technique handle high load factors? Is it compatible with dynamic resizing (rehashing)?
- **Memory Overhead**: Does the technique require extra memory, and how does that fit with your system's memory constraints?

Linear Probing: Simple but Prone to Clustering

Linear probing is straightforward to implement and understand. It offers good performance with low load factors but can suffer significantly from primary clustering as the load factor grows, leading to increased probing lengths.

Best for: Small datasets or applications where the load factor can be kept low.

Quadratic Probing: Reducing Clustering

Quadratic probing addresses the issue of primary clustering seen in linear probing by using a quadratic function to calculate probe sequences. However, it can suffer from secondary clustering and might not explore all table slots if not carefully implemented.

Best for: Applications where primary clustering is a concern and the table size is carefully managed to be prime.

Double Hashing: Minimizing Clustering and Maximizing Distribution

Double hashing minimizes both primary and secondary clustering by using a second hash function to calculate probe sequences. It's more complex to implement but offers excellent performance even at higher load factors.

Best for: Large datasets or performance-critical applications where the table is expected to be moderately to heavily loaded.

Making the Right Choice

Consider the following questions when choosing a collision resolution strategy:

- **What is the expected load factor of your hash table?**
    - Lower load factors (`< 0.5`) might work well with simpler methods like linear probing.
    - Higher load factors might necessitate more sophisticated methods like double hashing.
- **What is the distribution of your key space?**
    - If your keys are evenly distributed, linear or quadratic probing might suffice.
    - If your keys are prone to patterns that might cause clustering, consider double hashing.
- **What are your performance requirements?**
    - If your application is read-heavy, minimizing search time might be crucial, favoring techniques that reduce clustering.
    - If your application is write-heavy, consider the time complexities of insert and delete operations.
- **How critical is memory usage?**
    - Techniques like chaining (previous assignment) require extra memory for pointers or linked lists.
    - Open addressing techniques (linear, quadratic, double hashing) use the table more efficiently but need careful management to avoid performance degradation.

Choosing the right collision resolution strategy is a critical decision in the design of a hash table. It requires a careful evaluation of your application's requirements, key distribution, and performance needs. By understanding the trade-offs of each method, you can make an informed choice that ensures the efficiency and reliability of your hash table implementation. As with any data structure, testing and profiling with real-world data and use cases are essential to validate your choice and fine-tune the performance of your hash table.

Resolution Complexity

Analyzing Collision Resolution Complexity

In hash tables, the efficiency of collision resolution strategies is crucial for performance optimization. Understanding the Big O complexity of different strategies helps in predicting performance in various scenarios. In this section, we'll break down the Big O complexities of linear probing, quadratic probing, and double hashing, considering both average and worst-case scenarios.

Linear Probing

Average-Case Complexity:

- **Search**: $O(1)$ when the load factor is low. As the load factor increases and clustering occurs, the average complexity can deteriorate towards $O(n)$.
- **Insert**: $O(1)$ under the same conditions as search. Performance degrades similarly as the table gets more populated.
- **Delete**: $O(1)$ on average, but can also worsen towards $O(n)$ with a high load factor and clustering.

Worst-Case Complexity:

- All operations (search, insert, delete) can degrade to $O(n)$ in the worst case, particularly when the table is close to being full or if significant clustering has occurred.

Quadratic Probing

Average-Case Complexity:

- **Search**: $O(1)$ under low load factors. The quadratic nature of the probe sequence reduces clustering, maintaining better performance than linear probing as the load factor increases.
- **Insert**: $O(1)$ for similar reasons as search. Quadratic probing manages clustering better, offering more consistent performance.
- **Delete**: $O(1)$ on average. As with search and insert, the performance remains more stable than linear probing under increasing load factors.

Worst-Case Complexity:

- Similar to linear probing, the worst-case complexity for all operations is $O(n)$. However, the onset of the worst-case scenario is typically delayed compared to linear probing due to better management of clustering.

Double Hashing

Average-Case Complexity:

- **Search**: $O(1)$. Double hashing significantly reduces both primary and secondary clustering, maintaining excellent performance even at higher load factors.
- **Insert**: $O(1)$. The second hash function ensures a more uniform distribution of keys, preventing clustering and maintaining efficient performance.
- **Delete**: $O(1)$ on average. The reduced clustering and improved distribution contribute to consistent performance across operations.

Worst-Case Complexity:

- All operations have a worst-case complexity of $O(n)$. However, due to the reduced clustering effect, the worst-case scenario is much less likely to occur in practice compared to linear and quadratic probing.

While the theoretical average-case complexities for search, insert, and delete operations in hash tables with open addressing are $O(1)$, the actual performance can vary based on factors such as the load factor, the quality of the hash function(s), and the specific collision resolution strategy used. Linear probing is simple but prone to clustering, quadratic probing addresses some of the limitations of linear probing, and double hashing offers the best distribution and performance, especially under high load factors.

It's important to note that these complexities assume a good hash function that distributes keys uniformly. In practice, the choice of hash function and collision resolution strategy should be made based on the specific requirements of your application, including the distribution of your keys and performance needs. Testing and profiling with real data are essential steps to ensure that your hash table is optimized for its intended use case.

Formative Assessment 1

Formative assessment 2